

Quantifying Contamination in Programs

Computer Science Department
Federal University of Minas Gerais

Seminar Report
Topics in Computer Science: Information Theory

Pedro Ramos
pedroramos@dcc.ufmg.br

Abstract—This is the report of a seminar in Information Theory, a class ministered by Professor Mario Sergio Alvim, and offered to the Master of Computer Science Program at the Federal University of Minas Gerais. The seminar topic chosen addresses the problem of quantifying Contamination in a program while doing Tainted Flow Analysis. We discuss the concept of Contamination and its relation with Shannon’s Mutual Information in Noisy Channels. We then propose to compare the accuracy of information regarding the Contamination of a program using a more precise Alias Analysis in contrast to a less precise Alias Analysis by doing Taint Analysis on the same program.

I. INTRODUCTION

Measuring the security of a system has been focus of research in computer science for many decades. Nowadays, amidst a wide and diverse range of applications, and considering the growth of Internet, this topic has become even more critical. The measurement of a system’s security and integrity usually consists in finding a vulnerability in the system and modelling what would be the form of attack that could explore that same vulnerability. Many of security vulnerabilities are described as **Tainted Flow Attacks** [1]. Examples may include: SQL Injection, Cross-Site Scripting, Malicious File Inclusion, Unwanted Command Executions, Eval Injections and even file system attacks.

In the Tainted Flow Attack, an individual exploit potential leaks in the system through a public interface. It does so by exploring a vulnerability that allows an input to reach an output in the system without passing through sanitisers, i.e, checks in the path of the program that will make sure that the input is safe. To find that vulnerability, we should ask the question: is there a path, in the program, from an input that leads to a trusted output, without passing through any checks (sanitisers)? Answering that question involves modelling the problem of Tainted Flow Analysis. [1]

Techniques of measuring the information flow of a program have been studied for many years and are already settled [2], [3] and thus this is not the focus of this project. Instead, we aim to use a new metric called **Contamination** to measure the Contamination of a program which is a generalised Tainted Flow Analysis. Further, we’re interested in proving that, when we use a more precise Alias Analysis whilst doing our Tainted Flow Analysis, we’re able to get also a more precise information concerning the program’s Contamination.

The goal of this seminar is to tangle the concepts of Tainted Flow Analysis and Contamination in regard to Information

Theory. We also want to compare two different alias analyses by quantifying the Contamination in a program when running Tainted Flow Analysis on the program’s dependence graph yielded by the use of these two different alias analyses.

In the section 2 we explain further what Tainted Flow Analysis is, and then we discuss the role of Alias Analyses in improving its accuracy. In section 3 we approach a method of quantifying Contamination and how it can be used to measure the Contamination of a program system, besides discussing its relation to Taint Analysis and Shannon’s Mutual Information. Section 4 has a brief discussion of the challenges to be met and future work concerning the bid of measuring the Contamination of programs using dependence graphs. Finally in section 5 we wrap up the theme and what’s been discussed.

II. TAINTED FLOW ANALYSIS

As previous said in the introduction, a Tainted Flow Attack can be seen as an information flow attack. It is basically an exploitation of a subpath in the program from a source t to a sink t that does not encounter any calls to sanitising functions. Sanitising functions clean the possibly malicious data or warns about the potential threat. The source t is an input channel. It reads information from the user and passes it to the program. Sinks s are sensible output operations, it writes to the program’s output channel. Sanitisers are functions that protect the program. For example, they could be an *if* check that verifies whether the information coming from the input is malicious or not. If it’s malicious, the sanitiser cleans it before passing it ahead. It can do so by generating a warning, throwing an exception, removing the malicious data, etc.

In order to find these paths in the program, we need to build the *Program Dependence Graph* (PDG) [4], settle which instructions in the program are *source* instructions and which are *sink* instructions. That is, we need to define which instructions are sensible untrusted inputs (they can either carry a secret or merely be an input-reader, making them *untrusted* for they suffer interference from outside the program) and which are trusted outputs. We may also need to define which instructions perform the role of *sanitisers*, if there’s any. Then we’re able to analyse the PDG and check if there are any paths that lead from sources to sinks without being sanitised.

To better illustrate the Tainted Flow Attack, let us regard these 2 programs depicted in figures 1 and 2. In 1 we see a clear vulnerability from a source to a sink. The dependence graph for Program 1 is seen in figure 3. If we consider the input as a *source* and the critical operation, which is a trusted operation,

```

PROGRAM 1:
{
    secret = input()
    performCriticalOperation(secret)

    ret
}

```

Fig. 1: Program 1 - a clear tainted flow vulnerability.

```

PROGRAM 2:
{
    secret = input()
    x = check(secret)
    if (x not malicious)
    {
        performCriticalOperation(secret)
    }

    ret
}

```

Fig. 2: Program 2 - the vulnerability has been suppressed with a sanitiser.

as a *sink*, we see that there's not a sanitiser in the path between a source and a sink, thus making it a vulnerable program. On the other hand, if we look at the graph for program 2, depicted in figure 4, we see that the check acts as a sanitiser and avoids the rise of a tainted path between source and sink.

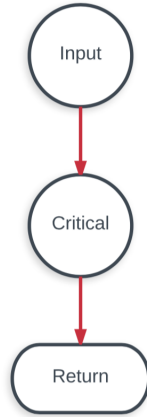


Fig. 3: Dependence Graph for Program 1.

A. Using Alias Analysis to improve Taint Analysis

In programming languages such as C/C++, that have a static memory model, sometimes it is hard to determine whether two pointers overlap during the course of the program's execution. So when we build the program's dependence graph of a program written in C or C++ we might deal with memory nodes with multiple loads and stores, and many different paths that pass through one same memory node. This happens

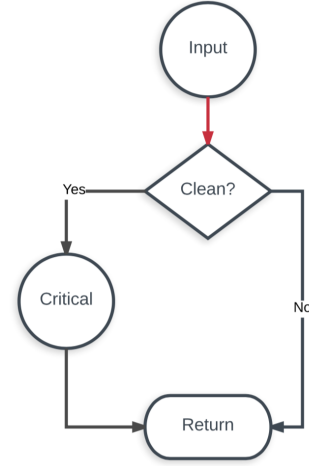


Fig. 4: Dependence Graph for Program 2.

because determining whether two pointers do not overlap is not a trivial task. In Compiler Theory, the task of doing so is called **Alias Analysis**. It is to determine whether two pointers are alias or not. There are many alias analyses nowadays, some of them are more precise than others. When we don't run any alias analyses on our program before we build its PDG, we end up with few memory nodes and a hugely imprecise graph. Consequently, when running a Tainted Flow Analysis on this PDG, we'll encounter many paths that are apparently tainted because they come from a source t , pass through one memory node, and this memory node happens to lead to a sink s .

The figure 5 represents a PDG generated with a very imprecise Alias Analysis. Even if there is only one sink node, once there's also only one memory node, every path that goes through this memory node is deemed tainted. But if we're able to disambiguate more pointers, we can have a PDG that is more precise, like in the example depicted in figure 6.

There we see that the mere presence of one more memory node reduced by 50% the number of tainted paths that lead to a trusted output. Hence, the aim of the proposed project hereby is to compare two different alias analyses while doing Tainted Flow Analysis. We aim to show that by using the Strict Relations Alias Analysis [5] we're able to get more precise information concerning the integrity of a program, given that we'll measure it through Taint Analysis in the program dependence graph.

However, in order to show that, we need a metric to measure how insecure a program can be, based on the Tainted Flow Analysis. We shall then address the concept of Contamination, proposed in [6] and then adapt the model of dependence graphs to be evaluated by this metric. We aim to use this measure of quantification to show that the Strict Relations Alias Analysis is a better Alias Analysis than the others Alias Analyses commonly used regarding the integrity and security of a program.

III. QUANTIFYING CONTAMINATION

To quantify the information conveyed in the tainted flows of a program, we should find a proper metric that addresses both

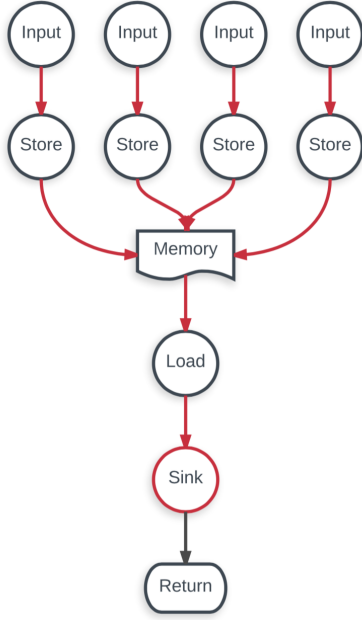


Fig. 5: A PDG with only one memory node, result of a very imprecise Alias Analysis.

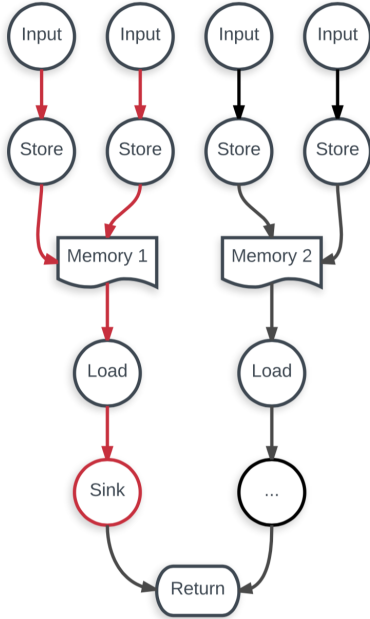


Fig. 6: A PDG with more memory nodes, result of a more precise Alias Analysis.

information flow and the program's integrity as a quantifiable concept. Ways of measuring the information flow of a program have been studied and extensively tackled [2], [3], [7]. In our case, we are interested in measuring the integrity of a program.

First off, it is needed that we define what Integrity means. As stated in [6], integrity is the prevention of unauthorized modification of information". We can consider that any damage

to the integrity of a program is a *corruption*. Said that, it is stated that there are typically two forms of corruptions in programs: *Contamination* and *Suppression*. Contamination occurs basically when there are tainted flows in the program, which is exactly what we're dealing with. Suppression, which will not be addressed in this seminar, measures the corruption done to the program's correctness.

Contamination is a generic form of Tainted Flow Analysis in a way that it tracks information flow from tainted inputs to outputs that are supposed to be untainted - or, using another terminology, from untrusted inputs to trusted outputs. This flow, as said in section 2, is a tainted flow. Trusted outputs are not supposed to be influenced by untrusted information, so in our case, if there's not a sanitiser in the path between a untrusted input to a trusted output, then this contaminated flow *corrupts the integrity of the program*. Thus, our main goal here is quantify the Contamination of a program.

We want to quantify the amount of information flows from untrusted inputs that contaminate trusted outputs. As said, this generalizes Taint Analysis, which qualitatively determines which untrusted inputs reach trusted outputs.

Definition. Contamination is the amount of information a user learns about untrusted inputs by observing trusted inputs and outputs.

The definition of Contamination requires two restrictions. First, the user may not directly read untrusted inputs. Otherwise, we would be quantifying something trivial—the amount of information the user learns about untrusted inputs by observing untrusted inputs. Second, the user may not read untrusted outputs, because we are interested only in the information the user learns from trusted outputs. These two restrictions result in the model seen in figure 7.

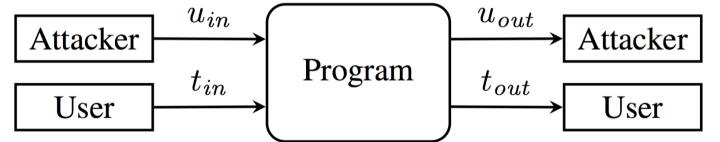


Fig. 7: Contamination model.

In Information Theory, we see the behaviour of channels. Channels, like a program, takes an input and returns an output. So we can view the program as an input. In the channel, the input has a probability distribution of individual events, and the channel is also able to turn randomness into defined outputs, so the outputs have also got a probability distribution of individual output events. In our scenario, let t_{in} , u_{in} , t_{out} be respectively trusted input, untrusted input and trusted output. Each event comprises of several input or output variables. Also let T_{in} , U_{in} , T_{out} be their respective probability distributions.

In Shannon's Information Theory, the *Mutual Information* characterises the quantity of information carried by channel inputs about the outputs by observing the outputs. Let $I(u_{in}, t_{out})$ denote the mutual information between events u_{in} and t_{out} , that is, the amount of information either event con-

veys about the other¹. Let $I(u_{in}, t_{out}|t_{in})$ be the conditional mutual information between untrusted input and trusted output, given that the input is trusted.

Hence we can say that the quantity C_1 of Contamination of trusted outputs by untrusted inputs in a single execution of the program is:

$$C_1 = I(u_{in}, t_{out}|t_{in})$$

To illustrate this metric, let us consider the following instruction as a single program.

$$o_t = i_u j_t$$

Suppose that variables o_t , i_u and j_t are one-bit trusted output, untrusted input, and trusted input, respectively, and that the values of i_u and j_t are randomly chosen. Intuitively, the user should be able to infer the value of i_u by observing j_t and o_t , hence there is 1 bit of Contamination. And according to the definition of C_1 , the quantity of Contamination caused by this program is indeed 1 bit.

Although [6] also defines the Contamination in a sequence of executions, in the expectation and the concept of leakage, we're mainly focused on a static analysis about the program, so what matters to us is the Contamination in a single execution, to be used as a metric to tell whether its integrity is too compromised or not.

IV. CHALLENGES AND FUTURE WORK

Now, given that we have thoroughly contemplated the concepts of Tainted Flow Analysis and the Contamination of a program in terms of Shannon's mutual information, we shall model our dependency graph to the Contamination model described in section 3. We need to find a way to describe the program dependence graph in terms of a distribution of untrusted inputs, trusted outputs and trusted inputs to be able to mathematically calculate the program's Contamination. After done that, we hope to compare two Alias Analysis: the Basic AA, implemented in the LLVM infrastructure and the SRAA, the Alias Analysis proposed in [5] in terms of which one yields programs that are deemed less contaminated due to the fact that they yield a more precise dependence graph regarding the memory model in it. The main challenge that can be faced in this work is the transition of the dependence graph to the Contamination model.

V. CONCLUSION

It has been very difficult to find a seminar topic for this module, once my field of studies is in compiler theory. This report describes a project proposal that couples Alias Analysis, Tainted Flow Analysis and information theory. More precisely, we aim to show that a better Alias Analysis can be very useful for many applications, and one of them is Tainted Flow Analysis. If we are able to produce a more accurate

Program Dependence Graph, with more unambiguous memory nodes, we are possible able to get more precise information concerning the integrity of the problem when running a Tainted Flow Analysis on it. In order to measure the improvement of accuracy in information, we need to find a way of measuring the integrity of a program. Thus we discussed the concepts of Information Theory present in [6].

There are typically two ways of measuring the corruption of a program - or its insecurity. We can measure its Contamination or its suppression. Here we were focused on measuring the Contamination only. We want to prove that a new Alias Analysis is able to improve Taint Analysis, and hence allow us to extract more information from the program in terms of its Contamination. Contamination is interested only in measuring the amount of contaminant in the output, i.e., the amount of tainted flows that lead to that trusted output. What the user does or not with the output is irrelevant; the user can ignore the fact that the information flow to the output can be tainted, and use only the trusted flow, however the output will still be contaminated. The Contamination is defined in terms of Shannon's Mutual Information and that's where Information Theory comes into play.

To continue the project, it is needed that we find a way of transforming the PDG in the Contamination model. That has not been done nor deepened in this work.

REFERENCES

- [1] A. Rimsa, M. d'Amorim, and F. M. Q. Pereira, "Tainted flow analysis on e-ssa-form programs," in *International Conference on Compiler Construction*. Springer, 2011, pp. 124–143.
- [2] D. Clark, S. Hunt, and P. Malacaria, "Quantitative information flow, relations and polymorphic types," *Journal of Logic and Computation*, vol. 15, no. 2, pp. 181–199, 2005.
- [3] D. E. Denning and P. J. Denning, "Certification of programs for secure information flow," *Communications of the ACM*, vol. 20, no. 7, pp. 504–513, 1977.
- [4] J. Ferrante, K. J. Ottenstein, and J. D. Warren, "The program dependence graph and its use in optimization," *ACM Transactions on Programming Languages and Systems (TOPLAS)*, vol. 9, no. 3, pp. 319–349, 1987.
- [5] M. Maroua, V. Paisante, P. Ramos, L. Gonnord, and F. Pereira, "Pointer disambiguation via strict inequalities," in *Proceedings of the 2017 International Symposium on Code Generation and Optimization*. ACM, 2017.
- [6] M. R. Clarkson and F. B. Schneider, "Quantification of integrity," *Mathematical Structures in Computer Science*, vol. 25, no. 02, pp. 207–258, 2015.
- [7] A. Sabelfeld and A. C. Myers, "Language-based information-flow security," *IEEE Journal on selected areas in communications*, vol. 21, no. 1, pp. 5–19, 2003.

¹Here it is used the notation $I(*, *)$ with a comma instead of a semicolon. This is because we want to emphasise that the relation is symmetric; it is not a mutual information between two distributions, but between two single events.